A new verifier for coding agents

Amadeo De La Vega
Department of Computer Science
University of Maryland
adelaveg@umd.edu

Ignacio Hernandez Bas
Department of Computer Science
University of Maryland
ihbas@umd.edu

Shayan Shabihi
Department of Computer Science
University of Maryland
shabihi@umd.edu

Abstract

Recently, Large Language Models (LLMs) have become valuable tools for assisting developers by generating code given natural-language instructions [1]. However, **they frequently generate incorrect code**[1] [2], [3]. Therefore, in order to maintain projects with high quality AI-generated code, it is crucial to automatically check its correctness [4], [5], a central problem in the field of Machine Learning for coding. In this project, we design a system (verifier harness) that checks the correctness of AI-generated code.

1 Introduction and Literature Review

Verifying that a given arbitrary program is correct is a non-trivial task. Indeed, even deciding that it halts for a particular input (known as the *Halting* problem) is an *undecidable* problem [6]. Thus, we shouldn't expect to have a perfect system. However, we can expect to have a robust one that exhaustively detects as many bugs as possible, as efficiently as possible.

This is why modern harnesses employ different verification layers to assess the correctness of AI-generated patches: *syntactic analysis, semantic/structural checks and dynamic testing*. This section covers the most prominent evaluation systems (2023-2025), where we discuss their main scope, their layers, their metrics, and their limitations to justify why our multi-layer verification harness innovates from the other existing systems.

1.1 SWE-bench (2023)

- **Scope**: In order to evaluate the capabilities of LLMs to generate correct code in real-world scenarios, SWE-bench was introduced. Specifically, given a set of GitHub repositories, each of them having a description of a code issue, SWE-bench tests the patches they generate to resolve such issues [7].
- **Verification Layers**: This system focuses on dynamic verification using unit tests already given in the repository. The only static verification that is performed consists on ensuring that the code must compile in order to run the unit tests.
- **Metrics**: A generated patch is considered successful if it passes all the provided unit tests. The benchmark scores are reported as the percentage of issues solved. Each task includes two test suites. **FAIL_TO_PASS**: tests that fail before applying the correct patch and should pass thereafter (ensuring that the issue or bug has been fixed), and **PASS_TO_PASS**: tests that have always passed before and should keep passing.

[1] I.e., code that does not do what it is supposed to do based on the developer's description.

- **Limitations**: The SWE-bench evaluation is only as good as the previously defined tests. If the testing suites are not defined properly, incorrect patches could pass all the tests. Even more, researchers have found recently that **sometimes they generate incorrect patches that are deemed as correct by SWE-bench, inflating its effectiveness.** [8].Also SWE bench does not provide deeper static or semantic analysis and some of the tasks were unsolvable with the resources given. OpenAI with SWE-bench Verified [9] tried fixing these issues with the creation of human-validated tests, but the core idea of validating only using unit tests remains.

1.2 Copilot Evaluation Harness (2024)

- **Scope**: Microsoft designed this evaluation harness [10] in order to test code-generating LLMs in realistic IDE (*Integrated Development Environments*) scenarios.
- **Verification Layers**: This system implements both static and dynamic checks adapted for each scenario.
- **Metrics**: The Copilot harness defines scenario-specific metrics. Common metrics include **Syntax Correctness** (percentage of outputs that are syntactically valid) and **Test Pass Rate** (percentage of test cases passed by the generated code). For bug fixes, a crucial metric is the **Fix Success Rate**, whether the targeted static analysis error was eliminated without new errors (essentially a pass/fail for that fix) . Documentation is scored on format/content correctness (e.g. presence of all required elements in the docstring) . The workspace query uses IR metrics like **MRR** (Mean Reciprocal Rank) and checks for correct keywords in answers. This harness thereby produces a richer set of metrics beyond simple pass/fail combining static quality checks (e.g. docstring format, absence of linter errors) with dynamic success rates (test case outcomes).
- **Limitations**: it serves mainly as a task-specific evaluation framework rather than a general-purpose patch verifier. It lacks deep semantic comparison between implementations and does not incorporate fuzzing, or a multi-layer integration. As a result, it may overlook logical errors or maintainability issues, focusing instead on predefined scenarios and static checks tied to each task.

1.3 PATCHDIFF (2025)

- **Scope**: PATCHDIFF is a testing framework that targets the verification of AI-generated patches by comparing them with developer (oracle) patches [8]. Unlike SWE-bench, which relies solely on predefined unit tests, PATCHDIFF actively generates tests that expose behavioral differences between the oracle patches and the AI-generated patches, allowing it to detect subtle functional errors that SWE-bench may miss.
- **Verification Layers**: it operates through a layered dynamic verification pipeline. First, it identifies functions impacted by a patch. Then, it extracts *contextual code* to narrow the scope of testing. An LLM is prompted to generate targeted tests designed to behave differently across the oracle and candidate patches. Finally, the system filters and reruns these tests to ensure stability, discarding irrelevant cases.
- **Metrics**: The framework reports several metrics. Most importantly, the **Suspicious Patch Rate** measures the percentage of *plausible patches* (i.e., patches that pass the SWE-bench tests) for which the PATCHDIFF-generated tests reveal behavioral discrepancies.
- **Limitations**: its effectiveness depends on the availability of oracle patches, making it unsuitable for scenarios without ground-truth patches. Its reliance on LLM-based generation incurs additional costs. Furthermore, while it improves coverage, it cannot guarantee exhaustive detection of all behavioral deviations, particularly in complex repositories.

1.4 AutoPatchBench (2025)

- **Scope**: *AutoPatchBench* [11] is a benchmark and evaluation framework developed by Meta AI to assess AI-based program repair for security vulnerabilities. Released as part of the *CyberSecEval* suite, it focuses on C/C++ programs containing real-world vulnerabilities discovered through fuzz testing. Each benchmark instance provides the original buggy code,

the fuzz input that triggers the crash, and the developer's ground-truth patch, allowing a standardized comparison of LLM-generated fixes against known-correct solutions.

- **Verification Layers**: AutoPatchBench employs a rigorous multi-layer verification pipeline, combining static, dynamic, and semantic checks:
  - *Static Verification*: Ensures that each candidate patch compiles successfully, validating basic syntactic and type correctness. Any patch that fails to build is immediately rejected.
  - *Dynamic Verification (Initial Test)*: Executes the patched program on the original crashing input to confirm that the vulnerability is fixed (i.e., the crash no longer occurs).
  - *Dynamic Verification (Extended Fuzzing)*: Subjects the patched program to further fuzz testing using the same fuzz harness that discovered the bug, evaluating robustness against similar edge cases.
  - *Semantic/Structural Verification*: Implements *white-box differential testing*, running the LLM-generated patch and the ground-truth patch side-by-side under identical inputs. Using debugger instrumentation, AutoPatchBench compares internal execution states—local variables, return values, and memory snapshots—at equivalent call-stack points. If both versions exhibit identical internal states across multiple fuzzed inputs, the patch is considered semantically equivalent to the developer's fix.
- **Metrics**: A patch is labeled as **correct** only if it passes all verification stages. The benchmark reports the overall **Patch Success Rate**, corresponding to the proportion of vulnerabilities fully fixed by the model. Intermediate statistics, such as the number of patches that compile, remove crashes, or survive fuzzing, are also recorded to analyze where failures occur.
- **Limitations**: it is domain-specific, it targets security-related C/C++ vulnerabilities and requires known crashing inputs and ground-truth patches for differential analysis. As such, it cannot evaluate general logical or semantic bugs in high-level languages (e.g., Python or Java). Additionally, while its differential testing provides strong guarantees of behavioral equivalence, it depends on runtime instrumentation and fuzzing, which are computationally intensive. The framework does not measure maintainability or test adequacy, focusing solely on functional correctness and robustness in the context of security patches.

1.5 Agentic Pipelines (2024–2025)

- **Scope**: Recent research has explored *agentic code-repair systems*, where LLMs act as autonomous agents capable of iteratively editing, testing, and refining code. These frameworks, such as *Trae Agent* (2025)[12] treat the LLM as a debugging agent that executes build commands, runs tests, and adjusts patches based on feedback. Their goal is to increase patch correctness through self-correction rather than evaluate existing patches.
- **Verification Layers**: Agentic pipelines naturally combine several verification layers within their iterative loop. Each generated patch undergoes a *static verification* step to ensure it compiles or passes syntax checks, followed by *dynamic testing* where the patch is run against predefined or generated unit tests. Some frameworks analyze *execution traces* or compare behavioral differences between runs, offering a heuristic form of *semantic analysis*. Although these systems do not perform formal AST or CFG comparisons, they implicitly reason about semantics through test outcomes and runtime feedback.
- **Metrics**: The main metric is the **Pass@1 success rate**—the percentage of issues successfully solved by the final patch. Some studies also report metrics such as the number of iterations required, the time-to-fix, or the **oracle success rate** (if the best patch among multiple candidates were chosen). These pipelines usually aim to maximize the proportion of successfully fixed issues rather than generate detailed verification reports.
- **Limitations**: Agentic repair frameworks focus on *producing* correct patches rather than *verifying* them comprehensively. They often rely on the existing test suite as the ultimate ground truth and do not include fuzzing, coverage analysis, or structured reporting.

1.6 Structural Innovation of our Harness

The comparative analysis of existing frameworks reveals that current evaluation systems address only isolated aspects of verification. **SWE-bench** focuses exclusively on dynamic correctness through unit tests, lacking static and semantic insight. The **Copilot Evaluation Harness** extends this with static analyzers but remains limited to task-specific IDE scenarios. Both **PATCHDIFF** and **AutoPatchBench** introduce a multi-layer pipeline with *differential* testing, though they rely on ground-truth patches. Finally, **Agentic Pipelines** demonstrate iterative patch generation but rely entirely on pre-existing tests as validation, without evaluating test suite quality or maintainability.

Our harness bridges these gaps by integrating four complementary layers into a unified framework:

- The **Static Verification Layer** identifies syntactic and structural anomalies before execution, leveraging ASTs, linters, and type checkers to ensure code quality and maintainability.
- The **Dynamic Verification Layer** evaluates functional correctness without ground truth patches by (in order):
  1. creating a sandboxed environment,
  2. running all the unit tests available,
  3. running all/selected *rule* tests,
  4. generating and running more tests using *fuzzing* to uncover hidden runtime errors,
  5. generating a final test that checks correctness with high probability (presumably), in case all previous tests have passed.
- The **Verification Report Layer** aggregates all previous results into a structured, multi-metric report summarizing static quality, and dynamic correctness.

Hence, our harness does not only provide a measure of whether a patch *passes*, but it also provides an understanding of both static and dynamic correctness, all within one cohesive system that **does not require ground truth patches**. In addition, this system is designed in a *modular way*. This means that the first two layers can be run independently, and each component of these layers can also be run independently for the convenience of the developer/LLM using the harness.

2 The biggest challenge: the Dynamic layer

There are many types of bugs. But the hardest to catch are *logical* bugs. These are precisely those that make programs output a wrong answer. Even worse, it is possible to have a program that outputs correct answers most of the times, but incorrect answers for few inputs. So how do we catch these? Well, we know a few important things about them. Firstly, we have access to its source code (including the change made, specified by the diff). And secondly, **they must pass all unit tests**.[2]

The fact that they pass all unit tests means that we have a large number of pairs of input/output's that are correct (i.e., we have a large number of samples of input/output's that correspond to a correct implementation). This, and the structure of its source code give us a lot of information to *infer* what a correct implementation should be. Or at least, some important properties that such implementation is likely to satisfy. These properties should be valid for all pairs of input/output's, or in other words, these properties should be **invariants** for all pairs of input/output's. We can also think about them as relationships between inputs and outputs that determine a correct implementation.

Indeed, one can prove correctness for many algorithms based on their invariants [13–15]. But how can we infer them? Well, there are many known techniques and already available software [16–18]. For this project, we have used SymInfer [19] [20]. This software is able to infer important invariants for arbitrary algorithms given a *trace*[3] of their variables in any given *location*. For instance, consider the following implementation of the division algorithm for two positive integers (Figure 1) and two of its traces at locations L1 and L2 (Figure 2):

[2] Except the extra generated fuzzing tests, since those only indicate a runtime error.

[3] Where trace is a sequence of numerical values of variables in a particular location of its program.

```
int cohendiv(int x, int y)
  assert(x >= 0 && y >= 1)
  int q=0; int r=x;
  while(r >= y){
    int a=1; int b=y;
    while[L1](r >= 2*b){
      a=2*a; b=2*b;
    }
    r=r-b; q=q+a;
  }
  [L2]
  return q;
```

Figure 1: Division algorithm to divide x by y

x	y	a	b	q	r
15	2	1	2	0	15
15	2	2	4	0	15
15	2	1	2	4	7
			⋮		
4	1	1	1	0	4
4	1	2	2	0	4
			⋮		

Figure 2: Traces for $(x, y) = (15, 2)$ and $(x, y) = (4, 1)$ respectively.

Then, using only the traces in L1 and L2 (i.e., only correct numerical values), SymInfer is able to infer sufficiently strong invariants that determine the semantics of the division algorithm. Indeed,

At the end, `SymInfer` returns at L1 (loop) invariants such as

$$x = qy + r; \quad ay = b; \quad b \leq x; \quad y \leq r; \quad 0 \leq q; \quad 1 \leq b; \quad 1 \leq y$$

and at L2 the (post-condition) invariants such as

$$x = qy + r; \quad r \leq y - 1; \quad 0 \leq r; \quad r \leq x$$

I.e., these invariants completely determine the algorithm! Now, it is important to note that these invariants are only *likely* to be satisfied by a correct implementation, since they are extracted from correct traces. However, the more pairs of correct input/output's we have, the more likely these invariants are satisfied by a correct implementation.

Using this idea, we can infer likely invariants for a correct implementation having traces made of pairs of input/output's given by the previously passing tests. Hence, the idea is testing the patch by generating other pairs of its input/output's and see how many invariants are violated by them. A large fraction of violated invariants would indicate a likely incorrect implementation, and a low fraction of violated invariants would indicate a likely correct implementation.

3 Results

3.1 Experiment 1: Computing the *SQI*-index (Static Analysis) for SWE-bench Verified

For this experiment, we computed the Static Quality Index (SQI) for many SWE-bench Verified instances to assess the quality of their ground truth patches.

SQI aggregates important Python *linters*, each targeting a different notion of "quality": Pylint performs *AST-based code-quality* and bug-pattern analysis, Flake8 checks *PEP8-style compliance* and lightweight correctness warnings, Mypy checks *type correctness* using type hints, and Bandit *flags common security anti-patterns*. Since they optimize for different goals (maintainability vs. formatting/readability vs. typing vs. security), their outputs are complementary rather than redundant.

Figure 3 presents a boxplot of the SQI achieved by the ground truth patches across the first ten instances of repositories including astropy, django, matplotlib, and scikit-learn. The barplot indicates that Mypy identifies the highest number of issues, suggesting that even ground truth patches within this selection exhibit challenges with type consistency. In contrast, if Flake8 scores were higher, it

would indicate a failure to adhere to standard Python formatting. Repositories such as matplotlib demonstrate a high SQI with low variance; indicating that these patches are not only functional but also consistently adhere to style guides, utilize correct types, and avoid overly complex logic.

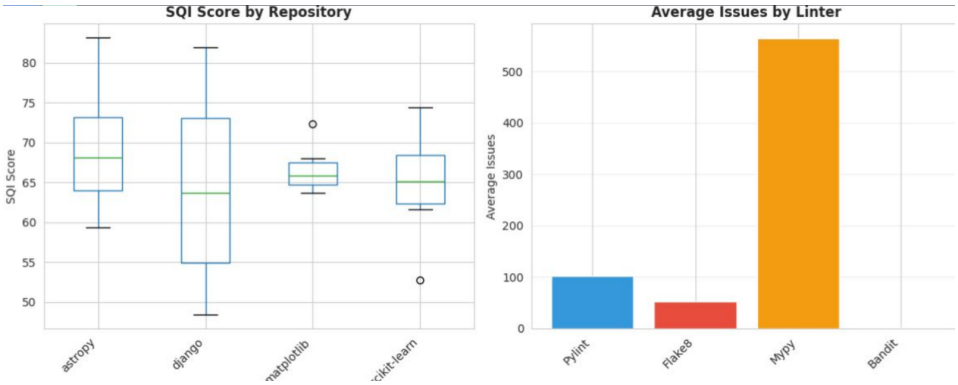

Figure 3: Syntax Quality Index scores for patches in SWE-bench-Verified

3.2 Experiment 2: Testing our container with SWE-bench Verified

In order to assess the validity of our container implementation, we generated patches using the isea_claude-3.5-sonnet agent for many SWE-bench Verified instances to compare the results of the baseline unit tests, both using SWE-bench and our implementation. If our implementation is correctly implemented, the results of SWE-bench compared to our implementation should be the same for all instances.

Indeed, our pipeline successfully pulled the required images and correctly executed the baseline unit tests for each repository: Table 3.2 demonstrates that all tests executed within our pipeline, using patches from the isea_claude-3.5-sonnet agent, produced results identical to the expected outcomes, confirming the reliability of our execution environment. (Here, **T=True**, **F=False**, **P=Positive**, **N=Negative**; meaning that **TP/TN** are the **Positive/Negative** SWE-bench results and **FP/FN** are our implementation **Positive/Negative** results that *differ* from SWE-bench. Since all **FP/FN** results are 0, then all our implementation results are the same as SWE-bench results).

In addition, our pipeline isolates and containerizes each instance, enabling parallel execution and significantly increasing system throughput, which is particularly advantageous for clustered environments. (We also adapted the existing infrastructure to transition from Docker to Podman containers).[4]

Repository	**Total**	**TP (✓)**	**TN (✓)**	**FP**	**FN**
django__django	114	69	45	0	0
scikit-learn__scikit-learn	16	8	8	0	0
pytest-dev__pytest	15	7	8	0	0
sphinx-doc__sphinx	9	2	7	0	0
astropy__astropy	6	3	3	0	0
pydata__xarray	5	1	4	0	0
mwaskom__seaborn	4	3	1	0	0
pallets__flask	3	0	3	0	0
TOTAL	**172**	**93**	**79**	**0**	**0**

Table 1: Unit test results executed per repository (*isea_claude-3.5-sonnet*)

3.3 Experiment 3: Testing the fuzzing implementation for run-time bugs with QuixBugs

QuixBugs is a dataset containing 40 short pairs of buggy/correct implementations of classic algorithms (each with a single-line bug) accompanied by failing/passing tests [21].

[4]Since neither Zaratan nor Nexus allow using Docker.

For this experiment, we applied our fuzzer on each buggy implementation.[5] Our fuzzer was able to detect all buggy implementations with run-time errors. I.e., all buggy implementations except those with logical bugs. To make it more efficient, we fuzzed only the variables that appeared on the diff.

Figure 4 shows: 1. (on the right) the type distribution of bugs in QuixBugs. 44.1% are run-time bugs that our fuzzer was able to detect and the rest (55.9%) are logical bugs (WrongOutput). 2. (on the left) the number of failures our fuzzer was able to generate. (We also included the number of "failures" we would get if we compare the output of the correct implementations with output of the buggy implementations, given the inputs our fuzzer generates. But this is extra information since we are assuming our pipeline does not have access to ground truth patches).

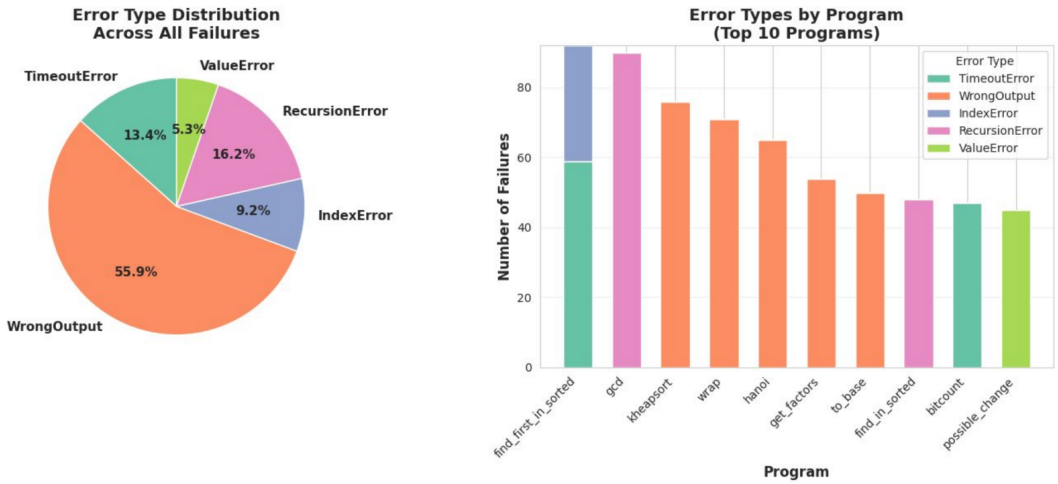

Figure 4: Error type distribution for QuixBugs and number of failures

3.4 Experiment 4: Testing our rules with SWE-bench Verified and QuixBugs

Using the taxonomy of bugs in [22] and their distribution (i.e., how often each type of bug appear in real life), we designed a few rules that intend to quickly catch bugs that follow specific patterns (explained in the following subsubsections). For this experiment, we run these tests on many instances of SWE-bench Verified and QuixBugs. Unfortunately, most rules were too weak to catch bugs; except rule 1 (it was useful for the fuzzer in the previous experiment to catch Timeout bugs). Of course, these rules are far from being exhaustive and more rules can be added with time.[6]

3.4.1 Rule 0: Change Detection (Did the patch really change code?)

Explanation: Sometimes a patch does not actually change the program. It might only change whitespace or comments, or it might accidentally re-add the same code as before. Rule 0 checks that the changed code is *meaningfully different* from the old code. If nothing actually changed, then the patch did not fix the bug it claimed to fix.

Example:

```
- if retries > 3:
-     backoff()
+ if retries > 3:
+     backoff()  # only whitespace/comment change
```

The patch looks like a change, but the program does the exact same thing.

[5] A fuzzer is a program that generates random inputs to feed another program to find hidden run-time errors.
[6] It is also possible that we did not choose a good dataset of programs to test them well.

3.4.2 Rule 1: Added Functions Must Run in Bounded Constant Time

Explanation: Rule 1 checks that new functions should not have loops whose number of iterations exceeds a constant (that the developer can choose and change easily), and it should not do unbounded recursion. This would try to catch infinite loops.

3.4.3 Rule 2: Definition–Use Execution

Explanation: Rule 2 checks that variables/resources are defined before they are used, and that resources that should be cleaned up (like files) actually get cleaned up on every path. Patches often add branches (`if/else`) or early returns and accidentally skip initialization or cleanup.

Example:

```
def read_data(flag):
    if flag:
        data = "ok"
    return data  #BUG: data is undefined when flag is False
```

This crashes only when `flag` is false.

3.4.4 Rule 3: Growing Resources

Explanation: Rule 3 runs a piece of code many times and watches for resources that keep growing. If a patch causes a leak (for example, adding items to a global list of "open" things and never removing them), the problem might not show up in one run but will show up after many runs.

Example:

```
OPEN_HANDLES = []

def allocate():
    handle = object()
    OPEN_HANDLES.append(handle) # BUG: never removed->leak over time
```

Repeated calls make `OPEN_HANDLES` grow forever.

3.4.5 Rule 4: Predicate Influence

Explanation: Rule 4 checks whether each part of a boolean condition actually matters. For example, in `A and B`, both `A` and `B` should be able to affect the final answer. If a patch accidentally duplicates a term or adds a constant like `and True`, one part may become "masked" (it no longer changes the outcome). The rule tries toggling inputs to see if each one can change the result.

Example:

```
# Intended:
return is_admin and has_token

# Buggy patch (has_token is ignored):
return is_admin and is_admin
```

Now `has_token` has no influence, so some cases are never tested correctly.

3.4.6 Rule 5: Robust Exception and Message Paths

Explanation: Rule 5 checks error-handling code. It flags patterns like "catch everything" (`except Exception`) and re-raising generic errors that lose useful information. It also looks for error messages/logs that are too vague to debug (missing an id/code/reason).

Example:

```
try:
    risky()
```

```
except Exception:
    raise Exception("failed")  # BUG: hides the real error cause
```

This makes debugging much harder, because the original error details are lost.

3.4.7 Rule 6: Structured Input and Conversion Validation

Explanation: Rule 6 checks input validation for structured data like dictionaries (think JSON). It tries a valid input, then tries inputs with missing fields and wrong types. Good code should accept valid inputs and reject invalid ones clearly (usually by raising an error).

Example:

```
EXPECTED_FIELDS = {"name": str, "count": int}

def parse(payload):
    # BUG: silently accepts missing fields
    return {"name": payload.get("name", ""),
            "count": payload.get("count", 0)}
```

Instead of failing fast, it hides problems by silently inserting defaults.

3.4.8 Rule 7: No Surprising Side Effects at Import Time

Explanation: In Python, importing a file runs all the top-level code in that file. Rule 7 checks that a patch does not accidentally add heavy or risky work at import time (like reading files, deleting files, or calling the network). Import-time side effects can make programs slow, flaky, or break in environments where a resource is missing.

Example:

```
# my_module.py (buggy):
import requests

# BUG: this runs during import, even if nobody calls a function yet
RESPONSE = requests.get("https://example.com/config").json()
```

Now `import my_module` can hang or fail if the network is down.

3.5 Experiment 5: Testing the likely-invariant idea for logical bugs with RunBugRun

RunBugRun is a dataset containing over 700,000 buggy/fixed programs accompanied by failing/passing unit tests. Most of the buggy implementations have only logical bugs [23].

For this experiment, we firstly selected 60 random buggy programs. Then, for each buggy/correct program, we computed its corresponding set of likely invariants based on its passing unit tests (i.e., based on its set of correct input/output's). Then, for each buggy/correct program, we selected other 100 random input/output's and computed the fraction of invariants violated for each input/output to compute the average fraction of violated invariants.

The following table shows these averages for each program.

Program	Avg (Correct)	Avg (Buggy)
p00001	0.033869	0.051202
p00007	1.000000	1.000000
p00009	0.275556	0.275433
p00024	0.025253	0.044872
p00026	0.000000	0.000000
p00027	0.005000	0.004630
p00032	0.018038	0.033766
p00035	1.000000	1.000000
p00037	0.962144	0.987926
p00039	0.666667	0.666667
p00044	0.400667	0.648889
p00063	0.101010	0.000000
p00066	0.065025	0.676136
p00112	0.036397	0.041235
p00117	0.353333	0.424000
p00123	0.000000	0.000000
p00135	0.508000	0.527027
p00139	0.873125	0.998087
p00147	0.146169	0.170564
p00148	0.538043	0.544118
p00183	0.706667	0.888889
p00251	0.932082	0.791798
p00254	0.000062	0.005291
p00311	0.041429	0.063616
p00331	0.254737	0.146479
p00332	1.000000	1.000000
p00333	0.020000	0.100000
p00353	0.000000	0.000000
p00356	0.253125	0.392857
p00378	0.000000	0.000000

Program	Avg (Correct)	Avg (Buggy)
p00385	0.063846	0.069339
p00439	0.260312	0.264060
p00473	0.082071	0.088141
p00474	0.001000	0.075000
p00490	0.000000	0.022222
p00491	0.264583	0.285714
p00499	0.046832	0.087542
p00589	0.804887	0.804907
p00590	0.588670	0.660150
p00670	0.002039	0.003631
p00803	0.008811	0.008889
p00935	0.048485	0.090323
p01063	0.009000	0.047368
p01155	0.000000	0.000000
p01349	0.147186	0.238095
p01447	0.000000	0.000000
p01479	0.990000	0.009804
p01601	0.215000	0.000000
p01636	0.000000	0.066667
p01722	0.005000	0.125000
p01751	0.142857	0.302381
p01789	0.071429	0.089286
p01852	0.005051	0.250000
p01968	0.963840	0.723631
p02233	0.745455	0.861905
p02235	0.398571	0.389325
p02256	0.020202	0.800000
p02263	0.885000	0.768333
p02274	0.865079	0.798733
p02305	0.000000	0.228571

We found that for the mayority of these programs (65%), Avg(Correct)$<$ Avg(Buggy). However, using only these averages, we cannot differentiate buggy from correct implementations in general.[7] Hence, this method alone does not work.

Nevertheless, there are a number of possible improvements to make. First, we only used numerical traces, which leads to having some *spurious* invariants (i.e., invariants that are true for the particular numerical values in the traces but not necessarily true in general for a correct implementation). For this, we can also implement a *symbolic-state* analysis using the source code of the buggy code (SymInfer admit symbolic-state analysis, but we did not use it because this is a bit more difficult to implement). Second, we only used traces that track correct input/output's. However, we can also track important "internal" traces, such as loop invariants (e.g., the traces of Fig 2). Third, during the search of likely invariants, we pruned the space of invariants in order to make this search more efficient. However, we can make this space of invariants dependent on the program at hand (possibly with a efficiency trade-off). Fourth, it is possible that the number of input/output's for each trace was not sufficiently large to find sufficiently strong invariants. And finally, it would be useful to find mathematical guarantees following the ideas in [16].

4 Conclusions and Future plans

The results of experiments 5.1-5.3 were satisfactory and will be kept in our pipeline. Unfortunately, experiments 5.4-5.5 were not. At this point it is not clear whether it is a good idea to keep adding rules or to completely discard them. However, we think it is worthwhile to improve the method of likely invariants given all the known techniques and results related to them [14, 16, 17, 19]. Another promising direction is to use random property-based testing.

In addition, we have the following future plans:

[7] E.g., we would like to obtain a global constant Ω such that, with high probability, Average$> \Omega$ for buggy implementations and Average$\leq \Omega$ for correct implementations.

- Making our GitHub repo (pipeline) available.
- Creating a webpage with pipeline in Streamlit so developers can easily verify their patches.
- Adding a module so developers can easily add their own rules.
- Adding an agent able to capture the intentions of developers.
- Adding *mutation testing* to our suite for validation.

References

[1] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pinto, Jared Kaplan, and Wojciech Zaremba. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[2] Emilia Hansson and Oliwer Ellréus. Code correctness and quality in the era of ai code generation: Examining chatgpt and github copilot, 2023.

[3] Chun Jie Chong, Zhihao Yao, and Iulian Neamtiu. Artificial-intelligence generated code considered harmful: A road map for secure and high-quality code generation. *arXiv preprint arXiv:2409.19182*, 2024.

[4] Aaron Councilman, David Fu, Aryan Gupta, Chengxiao Wang, David Grove, Yu-Xiong Wang, and Vikram Adve. Towards formal verification of llm-generated code from natural language prompts. *arXiv preprint arXiv:2507.13290*, 2025.

[5] Pranjal Aggarwal, Bryan Parno, and Sean Welleck. Alphaverus: Bootstrapping formally verified code generation through self-improving translation and treefinement. *arXiv preprint arXiv:2412.06176*, 2024.

[6] Leslie Burkholder. The halting problem. *ACM SIGACT News*, 18(3):48–60, 1987.

[7] Carlos E. Jimenez, Jui-Hsuan Yang, Alex Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *arXiv preprint arXiv:2310.06770*, 2023.

[8] You Wang, Michael Pradel, and Zhongxin Liu. Are "solved issues" in swe-bench really solved correctly? an empirical study. In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*, 2025. doi: 10.48550/arXiv.2503.15223.

[9] Neil Chowdhury, James Aung, Chan Jun Shern, Oliver Jaffe, Dane Sherburn, Giulio Starace, et al. Introducing swe-bench verified. `https://openai.com/index/introducing-swe-bench-verified/`, 2024. OpenAI Blog.

[10] Anisha Agarwal, Aaron Chan, Shubham Chandel, Jinu Jang, Shaun Miller, Roshanak Zilouchian Moghaddam, Yevhen Mohylevskyy, Neel Sundaresan, and Michele Tufano. Copilot evaluation harness: Evaluating llm-guided software programming, 2024. URL `https://arxiv.org/abs/2402.14261`.

[11] Meta AI Research. Introducing autopatchbench: A benchmark for ai-powered security fixes. `https://engineering.fb.com/2025/04/29/ai-research/autopatchbench-benchmark-ai-powered-security-fixes/`, 2025. Meta Engineering Blog.

[12] Trae Research Team, Pengfei Gao, Zhao Tian, Xiangxin Meng, Xinchen Wang, Ruida Hu, Yuanan Xiao, Yizhou Liu, Zhao Zhang, Junjie Chen, Cuiyun Gao, Yun Lin, Yingfei Xiong, Chao Peng, and Xia Liu. Trae agent: An llm-based agent for software engineering with test-time scaling, 2025. URL `https://arxiv.org/abs/2507.23370`.

[13] Robert W. Floyd. Assigning meanings to programs. In *Mathematical Aspects of Computer Science*, volume 19 of *Proceedings of Symposia in Applied Mathematics*, pages 19–32. American Mathematical Society, 1967.

[14] C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969. doi: 10.1145/363235.363259.

[15] Edsger W. Dijkstra. Guarded commands, nondeterminacy and formal derivation of programs. *Communications of the ACM*, 18(8):453–457, 1975. doi: 10.1145/360933.360975.

[16] Manuel Blum, Michael Luby, and Ronitt Rubinfeld. Self-testing/correcting with applications to numerical problems. In *Proceedings of the twenty-second annual ACM symposium on Theory of computing*, pages 73–83, 1990.

[17] Michael D. Ernst, Jake Cockrell, William G. Griswold, and David Notkin. Dynamically discovering likely program invariants to support program evolution. *IEEE Transactions on Software Engineering*, 27(2):99–123, 2001. doi: 10.1109/32.908957.

[18] Cormac Flanagan and K. Rustan M. Leino. Houdini, an annotation assistant for esc/java. In *FME 2001: Formal Methods for Increasing Software Productivity*, Lecture Notes in Computer Science, pages 500–517. Springer, 2001. doi: 10.1007/3-540-45251-6_29.

[19] Thanhvu Nguyen, KimHao Nguyen, and Matthew B Dwyer. Using symbolic states to infer numerical invariants. *IEEE Transactions on Software Engineering*, 48(10):3877–3899, 2021.

[20] ThanhVu Nguyen, KimHao Nguyen, and Hai Duong. Syminfer: inferring numerical invariants using symbolic states. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*, pages 197–201, 2022.

[21] Derrick Lin, James Koppel, Angela Chen, and Armando Solar-Lezama. Quixbugs: a multi-lingual program repair benchmark set based on the quixey challenge. In *Proceedings Companion of the 2017 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity*, SPLASH Companion 2017, page 55–56, New York, NY, USA, 2017. Association for Computing Machinery. ISBN 9781450355148. doi: 10.1145/3135932.3135941. URL `https://doi.org/10.1145/3135932.3135941`.

[22] Boris Beizer. *Software testing techniques*. dreamtech Press, 2003.

[23] Julian Aron Prenner and Romain Robbes. Runbugrun–an executable dataset for automated program repair. *arXiv preprint arXiv:2304.01102*, 2023.